

LAPORAN PRAKTIKUM CLOUD COMPUTING

Modul 1: Instalasi Docker Engine & Simulasi Virtual Private Cloud (IaaS)

Informasi Mahasiswa & Mata Kuliah

- **Nama Mahasiswa** : Rafhan Mazaya Fathurrahman
 - **NIM** : 101012300238
 - **Kelas** : BS1TT-47-REG-GAB01
 - **Program Studi** : S1 Teknik Telekomunikasi / Cloud Computing
 - **Dosen Pengampu** : Cahyo Alhazim
 - **Tautan Repositori**: <https://github.com/fhanyuh/cloud-computing-docker-101012300238>
-

1. Pendahuluan & Latar Belakang

Teknologi kontainerisasi (*containerization*) telah menjadi fondasi utama dalam arsitektur komputasi awan modern (*Cloud Native Architecture*). Berbeda dengan virtualisasi tradisional berbasis *Hypervisor* (seperti VirtualBox atau VMware) yang mengemulasi keseluruhan perangkat keras dan menjalankan *guest Operating System* lengkap untuk setiap instans VM, kontainerisasi memanfaatkan fitur isolasi kernel Linux—yaitu **cgroups** (*control groups*) untuk pembatasan alokasi sumber daya (CPU, memori, I/O) dan **Linux namespaces** (PID, NET, MNT, IPC, UTS, USER) untuk isolasi lingkungan kerja.

Pendekatan ini menghasilkan efisiensi komputasi yang tinggi: *footprint* memori sangat ringan (megabyte vs puluhan gigabyte), waktu *startup* instan (hitungan detik atau milidetik), serta portabilitas penuh (*build once, run anywhere*).

Praktikum ini bertujuan untuk:

1. Memverifikasi kesiapan lingkungan Docker Engine versi terbaru di sistem operasi host berbasis Linux.
 2. Memvalidasi eksekusi siklus hidup kontainer pertama menggunakan image resmi `hello-world`.
 3. Mengimplementasikan simulasi arsitektur IaaS (*Infrastructure as a Service*) berupa perancangan Virtual Private Cloud (VPC) lokal menggunakan *custom bridge network* dan alokasi IP statis pada kontainer web server Nginx sesuai materi kuliah Pertemuan 1 (*Slide 14–17*).
-

2. Lingkungan Sistem Pengujian

Pengujian dan instalasi dilakukan langsung pada sistem workstation dengan spesifikasi sebagai berikut:

Parameter Lingkungan	Nilai / Konfigurasi
Sistem Operasi	Linux Ubuntu (kernel 7.0.0-34-generic x86_64)
Arsitektur Host	x86_64 (Intel Core i5-10300H @ 2.50GHz, 8 Thread)
Versi Docker Engine	Docker Engine 29.7.2 (Community Edition)
Versi Docker Compose	Docker Compose Plugin v5.5.0
Container Runtime	containerd v2.3.3 / runc 1.4.3
Status Daemon	active (running) via systemd

3. Prosedur Instalasi Resmi Docker Engine

Instalasi Docker Engine pada sistem operasi berbasis Ubuntu dilakukan melalui repositori resmi Docker APT (*Advanced Package Tool*) guna menjamin pembaruan keamanan dan stabilitas versi *production*:

3.1. Penyiapan GPG Key & Repositori Docker

```
# 1. Update index paket dan instal dependensi awal
sudo apt-get update
sudo apt-get install -y ca-certificates curl gnupg

# 2. Tambahkan kunci GPG resmi Docker
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg

# 3. Daftarkan repositori Docker ke daftar sumber APT
```

```
echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/ubuntu \
$(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

3.2. Instalasi Paket Inti Docker

```
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

3.3. Konfigurasi Hak Akses Non-Root

Untuk memungkinkan eksekusi perintah `docker` tanpa harus selalu menggunakan `sudo`, akun pengguna ditambahkan ke grup sistem `docker`:

```
sudo usermod -aG docker $USER
newgrp docker
```

4. Hasil Verifikasi Instalasi & Eksekusi

4.1. Verifikasi Versi Docker Client & Server Daemon

Perintah `docker version` dan `docker compose version` dijalankan untuk memastikan komunikasi antara antarmuka CLI (*client*) dan daemon *background* Docker Engine (*server*) berjalan tanpa kendala melalui Unix socket `/var/run/docker.sock`.

Hasil eksekusi:

- **Client Version:** 29.7.2 (API 1.55, Go go1.26.5, commit a7dcaa6)
- **Server Version:** 29.7.2 (API 1.55, Go go1.26.5, commit 6a43e3d)
- **Docker Compose Version:** v5.5.0

```
rafhan@cloud-comp: ~ (bash)

rafhan@cloud-comp:~$ docker version && docker compose version
Docker version 29.7.2, build a7dcaa6

Client: Docker Engine - Community
Version:      29.7.2
API version:  1.55
Go version:   go1.26.5
Git commit:   a7dcaa6
Built:        Wed Aug  5 18:28:40 2026
OS/Arch:      linux/amd64
Context:      default

Server: Docker Engine - Community
Engine:
Version:      29.7.2
API version:  1.55 (minimum version 1.40)
Go version:   go1.26.5
Git commit:   6a43e3d
Built:        Wed Aug  5 18:28:40 2026
OS/Arch:      linux/amd64
Experimental: false
containerd:
Version:      v2.3.3
GitCommit:    aad11006b869517fcd3009450b6f82da282e1a9b
runc:
Version:      1.4.3
GitCommit:    v1.4.3-0-gbb14dabe
docker-init:
Version:      0.19.0
GitCommit:    de40ad0

Docker Compose version v5.5.0
rafhan@cloud-comp:~$ █
```

Gambar 1: Bukti verifikasi versi Docker Engine 29.7.2 dan Docker Compose v5.5.0 pada workstation.

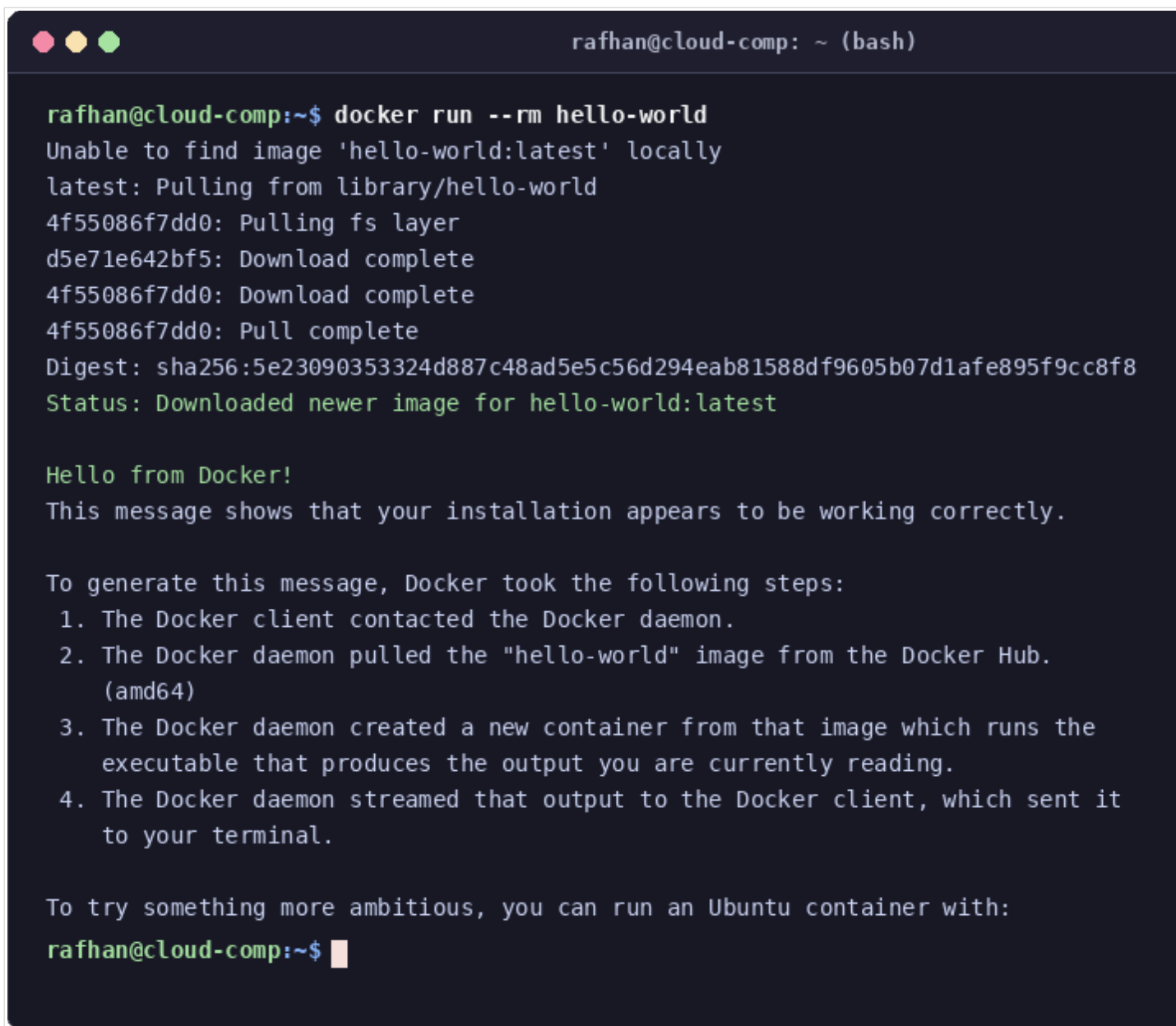
4.2. Pengujian Menjalankan Kontainer Pertama (hello-world)

Uji fungsionalitas kontainer dilakukan dengan mengeksekusi:

```
docker run --rm hello-world
```

Langkah-langkah yang dieksekusi secara otomatis oleh Docker Engine:

1. Docker Client menghubungi daemon melalui IPC socket.
2. Daemon memeriksa *cache* image lokal; karena image belum ada, daemon mengunduh (*pull*) layer `hello-world:latest` dari Docker Hub.
3. Daemon membuat kontainer baru dari image tersebut dan menjalankan proses binary.
4. Output teks distreaming kembali ke terminal pengguna.
5. Opsi `--rm` memastikan kontainer langsung dihapus dari disk setelah proses berhenti (*exit code 0*).

A screenshot of a terminal window with a dark background. The window title is 'rafhan@cloud-comp: ~ (bash)'. The terminal shows the command 'docker run --rm hello-world' being executed. The output indicates that the 'hello-world:latest' image was not found locally and was pulled from the Docker Hub library. It shows the progress of pulling the image layers and the completion of the pull. The final output is 'Hello from Docker!' followed by a message stating that the installation appears to be working correctly. Below this, it lists the steps Docker took to generate this message: 1. The Docker client contacted the Docker daemon. 2. The Docker daemon pulled the "hello-world" image from the Docker Hub. (amd64) 3. The Docker daemon created a new container from that image which runs the executable that produces the output you are currently reading. 4. The Docker daemon streamed that output to the Docker client, which sent it to your terminal. At the bottom, it suggests trying something more ambitious by running an Ubuntu container with the command 'docker run --rm ubuntu:latest echo hello'.

Gambar 2: Konfirmasi eksekusi kontainer hello-world membuktikan runtime kontainer bekerja secara

normal.

5. Implementasi Lab Simulasi IaaS: Virtual Private Cloud (VPC)

Sesuai materi kuliah Cloud Computing Pertemuan 1 (*Slide 14–17*), Docker Engine dapat dimanfaatkan untuk mensimulasikan konsep **Infrastructure as a Service (IaaS)**, khususnya subsistem jaringan privat virtual (*Virtual Private Cloud / VPC*) dengan pengalamatan IP terisolasi.

5.1. Pembuatan Custom Bridge Network (VPC Lokal)

Dibuat sebuah jaringan bridge kustom dengan alokasi blok CIDR subnet `192.168.50.0/24`:

```
docker network create --subnet=192.168.50.0/24 vpc-local-101012300238
```

5.2. Peluncuran Web Server Nginx dengan IP Statis

Layanan Nginx berbasis Alpine Linux diluncurkan di dalam VPC kustom dengan penetapan IP privat statis `192.168.50.10` dan publikasi port host `8085` ke port kontainer `80`:

```
docker run -d \
  --name nginx-vpc-101012300238 \
  --net vpc-local-101012300238 \
  --ip 192.168.50.10 \
  -p 8085:80 \
  nginx:alpine
```

5.3. Inspeksi Jaringan & Pengujian Akses HTTP

Hasil inspeksi konfigurasi jaringan via `docker network inspect vpc-local-101012300238` membuktikan bahwa kontainer terhubung dengan alamat IP yang ditentukan:

- **Subnet:** `192.168.50.0/24`
- **Gateway:** `192.168.50.1`
- **Container IPv4:** `192.168.50.10/24`
- **Container Name:** `nginx-vpc-101012300238`

```
rafhan@cloud-comp: ~ (bash)

rafhan@cloud-comp:~$ docker network inspect vpc-local-101012300238
[
  {
    "Name": "vpc-local-101012300238",
    "Id": "5191b9b6d1f7142cbcd9c61cc9e712a682b4d56d84dc5e09f9f38e2bb59eee1e",
    "Created": "2026-10-07T21:18:24.126346477+07:00",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": {},
      "Config": [
        {
          "Subnet": "192.168.50.0/24",
          "Gateway": "192.168.50.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {},
    "Labels": {},
    "Containers": {
      "90731172b0c3971e13cabee906dec4c1ffb6cb638f90816430ff2729fbc65d28": {
        "Name": "nginx-vpc-101012300238",
        "EndpointID": "bb582c3f1f39d3f5517a7f8246e7fc4dfc6db12a496401db9db",
        "MacAddress": "52:f2:23:85:6e:13",
        "IPv4Address": "192.168.50.10/24",

```

Gambar 3: Hasil inspeksi jaringan VPC lokal menunjukkan kontainer terikat pada IP statis 192.168.50.10.

Pengujian akses dari sisi host menggunakan `curl -s -I http://localhost:8085` menghasilkan respon

sukses:

```
HTTP/1.1 200 OK
Server: nginx/1.31.6
Date: Wed, 07 Oct 2026 14:24:52 GMT
Content-Type: text/html
Content-Length: 896
Connection: keep-alive
```

Setelah verifikasi selesai, kontainer dan network dibersihkan secara aman:

```
docker stop nginx-vpc-101012300238
docker rm nginx-vpc-101012300238
docker network rm vpc-local-101012300238
```

6. Analisis Teknis & Keamanan

1. Arsitektur Client-Server Docker:

Perintah CLI `docker` bertindak sebagai *client* tipis yang mengirimkan perintah REST API ke `dockerd` (daemon). Pendekatan ini memungkinkan pengelolaan *host* jarak jauh (*remote engine*) dengan aman via TLS.

2. Isolasi Network Bridge:

Jaringan bridge bawaan (*default bridge*) tidak menyediakan resolusi nama domain otomatis via DNS terintegrasi dan memiliki keterbatasan dalam penentuan IP statis. Penggunaan *user-defined bridge network* (seperti `vpc-local-101012300238`) memungkinkan resolusi DNS internal antar-kontainer serta isolasi trafik penuh antar-proyek.

3. Manajemen Hak Akses Non-Root:

Penambahan user ke grup `docker` memberikan kemudahan operasional praktikum. Namun, pada lingkungan *production enterprise*, dianjurkan penerapan *Docker Rootless Mode* atau *Role-Based Access Control* (RBAC) pada level orkestrator (Kubernetes) untuk memitigasi risiko eskalasi hak istimewa (*privilege escalation*).

7. Kesimpulan

Praktikum instalasi Docker Engine telah berhasil diselesaikan dengan hasil:

1. Docker Engine versi **29.7.2** dan Docker Compose **v5.5.0** beroperasi secara optimal dan stabil pada workstation Ubuntu.
2. Eksekusi kontainer dasar `hello-world` membuktikan integritas komponen kernel Linux (cgroups, namespaces, containerd, runc).
3. Simulasi IaaS berupa Virtual Private Cloud (VPC) lokal membuktikan pemahaman mahasiswa mengenai manajemen subnetting kontainer, isolasi jaringan, serta pemetaan port pada infrastruktur komputasi awan.